

Optimization

6.6 NSGA-II (multiobjective genetic algorithms)

José Firmino Aguilar Madeira

From the GA to NSGA-II: who wins the tournament?

6.5: a list built by polling. **6.6:** a **population that evolves**.

In the GA of 5.3, $f(x_a) < f(x_b)$ decided the tournament.

Now $f(x) = (f_1(x), \dots, f_m(x))$ and often

$$x_a \not\prec x_b \quad \text{and} \quad x_b \not\prec x_a.$$

From the GA to NSGA-II: who wins the tournament?

6.5: a list built by polling. **6.6:** a population that evolves.

In the GA of 5.3, $f(x_a) < f(x_b)$ decided the tournament.

Now $f(x) = (f_1(x), \dots, f_m(x))$ and often

$$x_a \not\prec x_b \quad \text{and} \quad x_b \not\prec x_a.$$

Who wins the tournament?

How do we do selection in a GA when there is no scalar fitness?

(Deb, Pratap, Agarwal and Meyarivan, 2002)

The answer has two ingredients

1. Convergence

Prefer those on a **better front** of the population:

dominance rank

2. Diversity

Among solutions equally good by dominance, prefer those in **less crowded regions**:

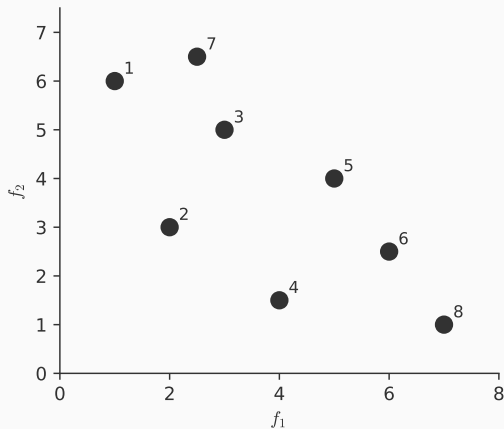
crowding distance

$\text{NSGA-II} = \text{rank} + \text{crowding}$

The two goals of 6.2 have become the **two selection criteria**.

Nondominated sorting: the rank

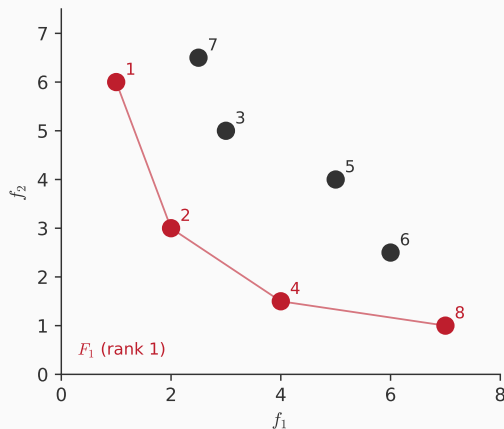
Eight solutions. Which are nondominated?



Nondominated sorting: the rank

Eight solutions. Nobody dominates 1, 2, 4 and 8:

$$F_1 = \{1, 2, 4, 8\} \quad (\text{rank 1}).$$



Nondominated sorting: the rank

Eight solutions. Nobody dominates 1, 2, 4 and 8:

$$F_1 = \{1, 2, 4, 8\} \quad (\text{rank 1}).$$

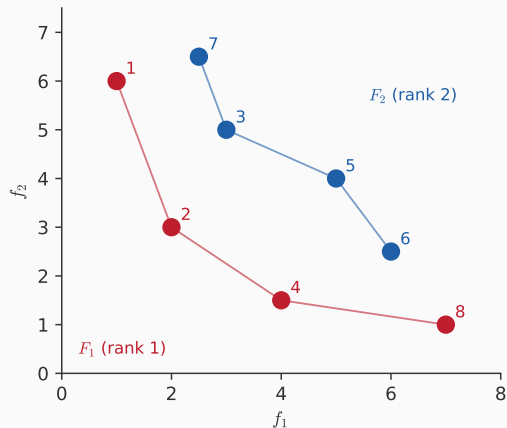
With these removed, repeat on the rest:

$$F_2 = \{3, 5, 6, 7\} \quad (\text{rank 2}).$$

rank 1 is better than rank 2

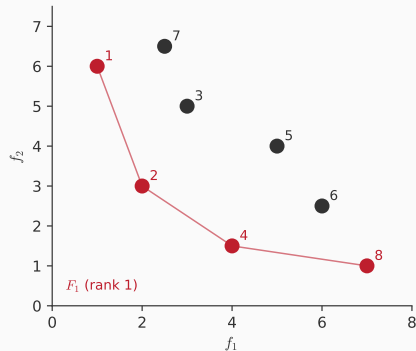
Caution

F_1 is the first front **of the current population** — it is not necessarily the Pareto front Y_N (6.2).



Crowding distance

There is room for only 3 of the 4 points of F_1 . By dominance, 1, 2, 4 and 8 are indistinguishable. Which one leaves?



Crowding distance

There is room for only 3 of the 4 points of F_1 . By dominance, 1, 2, 4 and 8 are indistinguishable. Which one leaves?

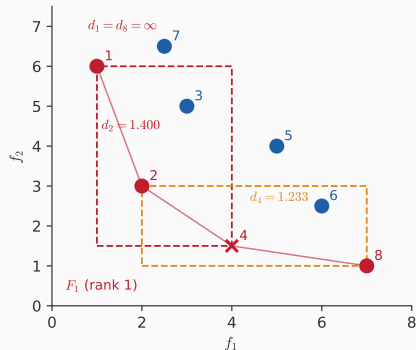
We want to preserve **diversity**: the one in a more crowded region leaves. The extremes get $d = \infty$; for the others, sorting by each objective,

$$d_p = \sum_i \frac{f_i(\text{next}) - f_i(\text{previous})}{f_i^{\max} - f_i^{\min}}.$$

Point 2 (neighbours 1 and 4):

$$d_2 = \frac{4-1}{7-1} + \frac{6-1.5}{6-1} = 0.5 + 0.9 = 1.4.$$

Point 4: $d_4 = 1.233$.



Crowding distance

There is room for only 3 of the 4 points of F_1 . By dominance, 1, 2, 4 and 8 are indistinguishable. Which one leaves?

We want to preserve **diversity**: the one in a more crowded region leaves. The extremes get $d = \infty$; for the others, sorting by each objective,

$$d_p = \sum_i \frac{f_i(\text{next}) - f_i(\text{previous})}{f_i^{\max} - f_i^{\min}}.$$

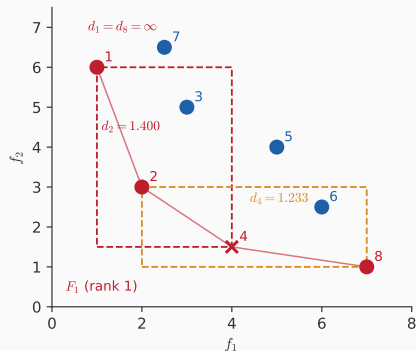
Point 2 (neighbours 1 and 4):

$$d_2 = \frac{4-1}{7-1} + \frac{6-1.5}{6-1} = 0.5 + 0.9 = 1.4.$$

Point 4: $d_4 = 1.233$.

1, 2 and 8 stay; 4 leaves.

larger d = less crowded region = higher priority



Crowded-comparison operator $\prec_n$

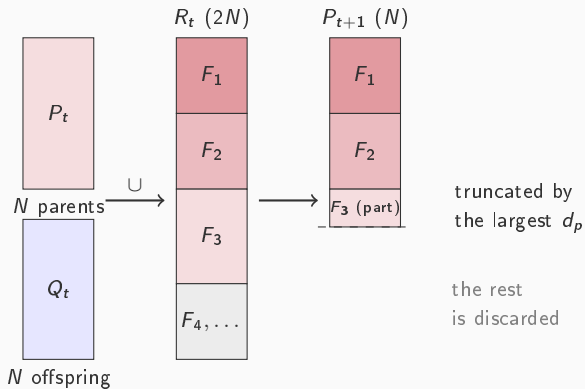
$$p \prec_n q \quad \text{if} \quad \text{rank}(p) < \text{rank}(q) \quad \text{or} \quad (\text{rank}(p) = \text{rank}(q) \text{ and } d_p > d_q).$$

Convergence first; in a tie, diversity.

This is how the binary tournament is decided.

It is not a new dominance relation: it is a tie-breaking rule for selection.

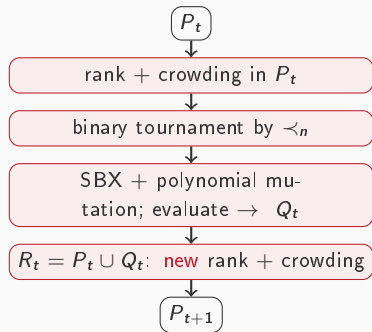
Elitism: parents and offspring compete



1. The N parents P_t generate N offspring Q_t .
2. Everything is merged:
 $R_t = P_t \cup Q_t$, with $2N$ solutions.
3. R_t is sorted into fronts $F_1, F_2, \dots$
4. P_{t+1} is filled front by front, while they fit whole.
5. The first one that does not fit is truncated by crowding.

The best parents can survive directly: **elitism**.

One generation of NSGA-II



Mating selection

Who generates offspring? Tournament: **stochastic**.

Environmental selection

Who survives? Truncation of $P_t \cup Q_t$: **deterministic**, given P_t and Q_t .

Elitism lies in the second.

The algorithm

NSGA-II: $N, G, p_c, p_m \leftarrow P_0 \leftarrow N$ random points in the box $[x^{(L)}, x^{(U)}]$

```
1: evaluate  $f$  on  $P_0$ 
2: for  $t = 0, \dots, G - 1$  do
3:   sort  $P_t$  into fronts; compute  $d_p$  in each front
4:    $Q_t \leftarrow N$  offspring: binary tournament by  $\prec_n$ , SBX, polynomial
    mutation; evaluate
5:    $R_t \leftarrow P_t \cup Q_t$ ; new sorting into fronts and new  $d_p$ 
6:    $P_{t+1} \leftarrow F_1 \cup F_2 \cup \dots$  while they fit; truncate the first one that
    does not fit by the largest  $d_p$ 
7: end for
8: return  $F_1$  of  $P_G$ 
```

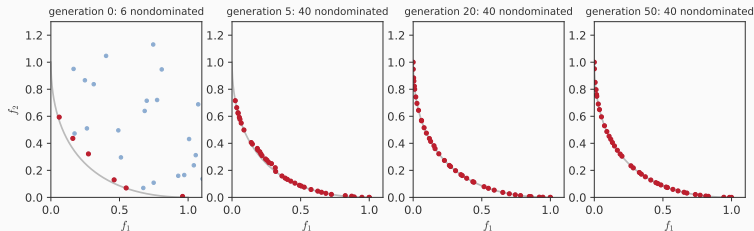
SBX and polynomial mutation: Deb's real-coded operators ($\eta_c \approx 15\text{--}20$, $\eta_m \approx 20$, $p_m = 1/n$), instead of the BLX and Gaussian mutation of 5.3: offspring close to the parents, spread controlled by η_c, η_m .

Cost: N evaluations per generation + $O(mN^2)$ for the sorting. If evaluating f requires a simulation (finite elements, CFD), the evaluations usually dominate — and the N of a generation are **parallelizable**.

What if there are constraints?

Deb's rule: feasible beats infeasible; between infeasible, smaller violation; between feasible, ordinary dominance. **No penalty to tune** (4.3.1).

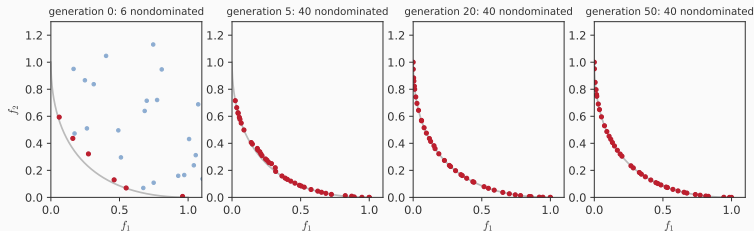
NSGA-II in action: generations 0 to 50



The course's convex example, $N = 40$. Generation 0: random population, 6 nondominated. Generation 5: almost all close to the front, poorly distributed. Generation 20: good coverage. Generation 50: refinement — HV 1.637 (exact 1.648; $r = (1.1, 1.65)$), 2040 evaluations.

Our own teaching implementation; the constraint $x_1^2 + x_2^2 \leq 1$ handled by Deb's rule, as above.

NSGA-II in action: generations 0 to 50

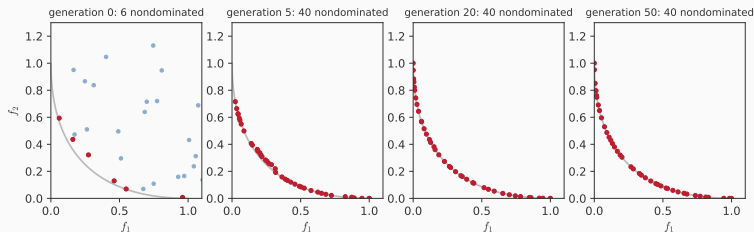


The course's convex example, $N = 40$. Generation 0: random population, 6 nondominated. Generation 5: almost all close to the front, poorly distributed. Generation 20: good coverage. Generation 50: refinement — HV 1.637 (exact 1.648; $r = (1.1, 1.65)$), 2040 evaluations.

Our own teaching implementation; the constraint $x_1^2 + x_2^2 \leq 1$ handled by Deb's rule, as above.

What is the rank doing? Bringing the population closer to the front.
And the crowding? Distributing it along the front.

NSGA-II in action: generations 0 to 50



The course's convex example, $N = 40$. Generation 0: random population, 6 nondominated. Generation 5: almost all close to the front, poorly distributed. Generation 20: good coverage. Generation 50: refinement — HV 1.637 (exact 1.648; $r = (1.1, 1.65)$), 2040 evaluations.

Our own teaching implementation; the constraint $x_1^2 + x_2^2 \leq 1$ handled by Deb's rule, as above.

What is the rank doing? Bringing the population closer to the front.

And the crowding? Distributing it along the front.

First **reach the front**, then **spread along it**: the two goals of 6.2.

On the computer: MATLAB and Python

MATLAB (Global Optimization Toolbox)

```
[x,fval] = gamultiobj(fun, nvars, ...  
    [],[],[],[], lb, ub, nonlcon);
```

`gamultiobj` is a **variant** of the NSGA-II family: there is no need to implement sorting, crowding or SBX to solve a problem.

Python (pymoo)

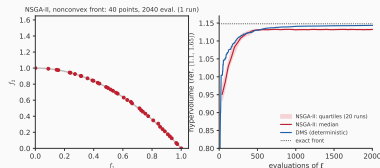
```
from pymoo.algorithms.moo.nsga2 import NSGA2  
from pymoo.optimize import minimize  
  
algorithm = NSGA2(pop_size=40)  
res = minimize(problem, algorithm,  
    ('n_gen', 50), seed=1)  
res.X, res.F      # decisions and objectives of the final approximation
```

$\text{pop_size} = N$ $\text{n_gen} = G$
 res.X : solutions res.F : objectives

Here NSGA-II is requested explicitly. Replace NSGA2 with NSGA3 or MOEAD: the problem stays the same; the strategy for convergence + diversity changes. `seed=1`: reproducible (3.2.1, 5.2, 5.3).

Course companion code: `nsga2_exemplo_uc.py` (convex example). The script prints: 40 points, $\text{n_eval} = 2000$ (in pymoo the initial population is the 1st of the 50 generations), HV 1.636 ($r = (1.1, 1.65)$). Our implementation (initial + 50 generations): 2040 evaluations, HV 1.637.

DMS vs. NSGA-II, on the same problem



budget n_f	DMS	NSGA-II med.	[Q1, Q3]
100	1.064	0.986	[0.968, 1.006]
200	1.094	1.083	[1.066, 1.096]
400	1.123	1.125	[1.117, 1.129]
800	1.139	1.132	[1.131, 1.133]
2000	1.144	1.132	[1.132, 1.133]

- Nonconvex example: NSGA-II also captures *non-supported* points — it uses dominance, not weights.
- **Same indicator, reference and budget:** DMS as one curve (deterministic); NSGA-II over 20 runs, median and quartiles. Exact front: 1.148 ($r = (1.1, 1.65)$). Each budget uses the last complete NSGA-II generation that does not exceed it.
- At 100 evaluations DMS(1) is ahead; at 200–400 they are tied; then NSGA-II levels off near 1.132 and DMS keeps going (1.144).
- **On this problem, with these parameters and this budget.** On other problems (rugged, fronts in several pieces) the population may be an advantage.

NSGA-II

$$|P_t| = N$$

The final approximation has at most N points: the **resolution** of the representation of the front is limited by the population (here, $N = 40$: it levels off near HV 1.132).

DMS

$$|L_k| \text{ can grow}$$

The list gains points as the poll step refines: the resolution **increases with the budget**.

A **structural** difference — more interesting than “which one won”.

Advantages

- many points in one run;
- *supported* and *non-supported*;
- does not require derivatives;
- flexible: different representations and operators make it possible to handle continuous and discrete variables, permutations and constraints;
- parallelizable evaluations;
- mature libraries.

Limitations

- many evaluations;
- stochastic $\rightarrow$ several runs;
- several parameters ($N, p_c, p_m, \eta_c, \eta_m$);
- fixed population limits the resolution;
- no guarantee of the global front with a finite budget;
- with many objectives, dominance loses discriminating power.

Other multiobjective evolutionary algorithms

The problem is always the same: **how to create pressure to converge and, at the same time, maintain diversity?**

NSGA-II (2002)	rank + crowding
MOEA/D (2007)	decomposition into scalar subproblems (Tchebycheff, 6.3)
NSGA-III (2014)	reference points — designed for many objectives

Another classic: SPEA2 (2001), with fitness based on dominance “strength” and an external archive.

Summary table — Chapter 6

	weighted sum	ε -constraint	DMS	NSGA-II
one point or a front?	one (per w)	one (per ε)	front	front
needs derivatives?	those of the solver	those of the solver	no	no
nonconvex front?	only <i>supported</i>	yes*	yes	yes
cost	one full optimization per point	one full optimization per point	up to $2n$ evaluations of f per poll step	N evaluations per generation
deterministic?	if the solver is	if the solver is	in the version presented	no (several runs)
control of position guarantee	weak efficient ($w_i > 0$, global minimizer)	direct unique $\Rightarrow$ efficient; any efficient point recoverable*	— Pareto–Clarke critical limit point (Lipschitz, dense directions)	— none with a finite budget

* with suitable ε and a global solution of the subproblem. For study. In class: the next two pages.

KEY TAKEAWAYS — 6.6

1. NSGA-II is a multiobjective GA: dominance rank + crowding replace the scalar fitness.
2. Rank $\rightarrow$ **convergence**; crowding $\rightarrow$ **diversity**.
3. **Elitism** comes from $P_t \cup Q_t$ and from the choice of the N survivors.
4. One run gives many points, including *non-supported* ones, but it is stochastic: **several runs, median and quartiles** to compare.
5. Cost: N evaluations per generation; the fixed population limits the resolution and, with many objectives, dominance loses selective power.

KEY TAKEAWAYS — Chapter 6

several objectives $\Rightarrow$ in general, **there is no single best solution**

dominance $\Rightarrow$ a **set of trade-offs** (the Pareto front)

good approximation = **convergence** + **diversity**

one point at a time (6.3, 6.4) or many points in one run (6.5, 6.6)

optimization $\rightarrow$ **trade-offs** $\rightarrow$ **decision**

And so the course closes: from a minimum of a function of one variable (Ch. 2) to a set of trade-offs between several objectives (Ch. 6) — always with the same questions: what information about f do we have, how much does it cost, what is guaranteed.

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Deb, K., Pratap, A., Agarwal, S., Meyarivan, T., *A fast and elitist multiobjective genetic algorithm: NSGA-II*, IEEE Transactions on Evolutionary Computation 6(2):182–197, 2002 — the NSGA-II paper.
- Deb, K., *Multi-Objective Optimization Using Evolutionary Algorithms*, Wiley, 2001.
- Madeira, J. F. A., *Optimização multi-objectivo da topologia de estruturas com algoritmos genéticos*, PhD thesis in Mechanical Engineering, IST, Universidade Técnica de Lisboa, 2004 — multiobjective GA, in Portuguese.
- Deb, K., *An efficient constraint handling method for genetic algorithms*, Computer Methods in Applied Mechanics and Engineering 186(2–4):311–338, 2000 — Deb's rule.

Resources

- Companion code: `nsga2_exemplo_uc.py` (pymoo). MATLAB: `gamultiobj` (Global Optimization Toolbox).
- Python (pymoo): `NSGA2(pop_size=...), minimize(problem, algorithm, ('n_gen', G), seed=1), res.X, res.F`; alternatives NSGA3, MOEAD.